



PROLOG

Tayná Fischer
Vander Alves

Brasília, 2026



O Paradigma Lógico

- Programas são relações entre E/S
- Estilo declarativo, como no paradigma funcional
- Na prática, inclui características imperativas, por questão de eficiência
- Aplicações: prototipação em geral, sistemas especialistas, banco de dados, ...

Modelo Computacional do Paradigma Lógico





Visão Crítica do Paradigma Lógico

→ Vantagens

Em princípio, todas do paradigma funcional

Permite concepção da aplicação em um alto nível de abstração (através de associações entre E/S)

→ Problemas

Em princípio, todos do paradigma funcional

Linguagens usualmente não possuem tipos, nem são de alta ordem



Silogismos

- Silogismo é um modelo de raciocínio baseado na ideia da dedução, composto por duas premissas que geram uma conclusão.
- Todos os homens são mortais.
- Sócrates é homem,
- Logo, Sócrates é mortal



Silogismos

- Todos os homens são mortais.
- Sócrates é homem,
- Logo, Sócrates é mortal



PROLOG

- PROgramming in LOGic
- A linguagem Prolog surgiu na década de 70
- Linguagem de paradigma lógico
- Linguagem declarativa baseada nos princípios da lógica
 - ◆ Nós dizemos ao programa QUAL problema queremos resolver
- É uma linguagem de uso geral que é especialmente associada com a inteligência artificial e linguística computacional
- De sintaxe simples, consiste numa linguagem puramente lógica com componentes extra-lógicos

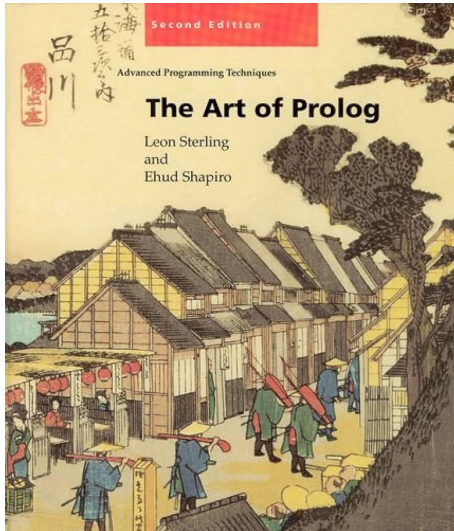


Aplicações de PROLOG

- Engenharia de Software (especificação formal, linguagens de programação);
- IA (raciocínio e Representação do Conhecimento), sistemas especialistas;
- Banco de Dados - Dedutivos (BDDs);
- Teoria Lógica (TL) das provas;
- Jogos;
- Lingüística Computacional;
- Processamento de linguagem natural;



Bibliografia



→ **The Art of Prolog: advanced programming techniques - (2nd edition).**
L. Sterling & E. Shapiro, MIT Press, 1994.



Como utilizar?

→ Editores online

<https://swish.swi-prolog.org/>

→ Download

<https://www.swi-prolog.org/download/stable>

tutorial: <https://www.swi-prolog.org/pldoc/man?section=quickstart>



Exemplo

```
1 pai(joão, pedro).
2 pai(joão, henrique).
3 pai(pedro, clara).
4 pai(pedro, clarissa).
5
6 mãe(maria, pedro).
7 mãe(maria, henrique).
8 mãe(luisa, clara).
9 mãe(luisa, clarissa).
10
11 homem(joão).
12 homem(pedro).
13 homem(henrique).
14
15 mulher(clara).
16 mulher(clarissa).
17 mulher(luisa).
```

```
19 filho(X, Y) :- pai(Y, X), homem(X).
20 filho(X, Y) :- mãe(Y, X), homem(X).
21
22 filha(X, Y) :- pai(Y, X), mulher(X).
23 filha(X, Y) :- mãe(Y, X), mulher(X).
24
25 avô(X, Y) :- pai(X, Z), pai(Z, Y).
26 avô(X, Y) :- pai(X, Z), mãe(Z, Y).
27
28 avó(X, Y) :- mãe(X, Z), pai(Z, Y).
29 avó(X, Y) :- mãe(X, Z), mãe(Z, Y).
```



Exemplo

```
1 pai(joão, pedro).
2 pai(joão, henrique).
3 pai(pedro, clara).
4 pai(pedro, clarissa).
5
6 mãe(maria, pedro).
7 mãe(maria, henrique).
8 mãe(luisa, clara).
9 mãe(luisa, clarissa).
10
11 homem(joão).
12 homem(pedro).
13 homem(henrique).
14
15 mulher(clara).
16 mulher(clarissa).
17 mulher(luisa).
```



Fatos

```
19 filho(X, Y) :- pai(Y, X), homem(X).
20 filho(X, Y) :- mãe(Y, X), homem(X).
21
22 filha(X, Y) :- pai(Y, X), mulher(X).
23 filha(X, Y) :- mãe(Y, X), mulher(X).
24
25 avô(X, Y) :- pai(X, Z), pai(Z, Y).
26 avô(X, Y) :- pai(X, Z), mãe(Z, Y).
27
28 avó(X, Y) :- mãe(X, Z), pai(Z, Y).
29 avó(X, Y) :- mãe(X, Z), mãe(Z, Y).
```



Exemplo

```
1 pai(joão, pedro).
2 pai(joão, henrique).
3 pai(pedro, clara).
4 pai(pedro, clarissa).
5
6 mãe(maria, pedro).
7 mãe(maria, henrique).
8 mãe(luisa, clara).
9 mãe(luisa, clarissa).
10
11 homem(joão).
12 homem(pedro).
13 homem(henrique).
14
15 mulher(clara).
16 mulher(clarissa).
17 mulher(luisa).
```

```
19 filho(X, Y) :- pai(Y, X), homem(X).
20 filho(X, Y) :- mãe(Y, X), homem(X).
21
22 filha(X, Y) :- pai(Y, X), mulher(X).
23 filha(X, Y) :- mãe(Y, X), mulher(X).
24
25 avô(X, Y) :- pai(X, Z), pai(Z, Y).
26 avô(X, Y) :- pai(X, Z), mãe(Z, Y).
27
28 avó(X, Y) :- mãe(X, Z), pai(Z, Y).
29 avó(X, Y) :- mãe(X, Z), mãe(Z, Y).
```



Regras



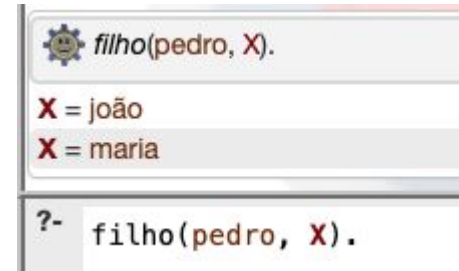
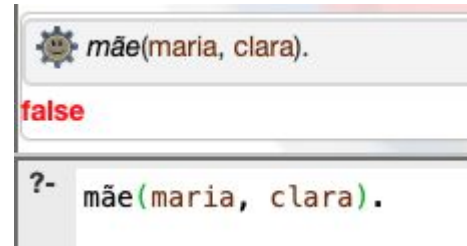
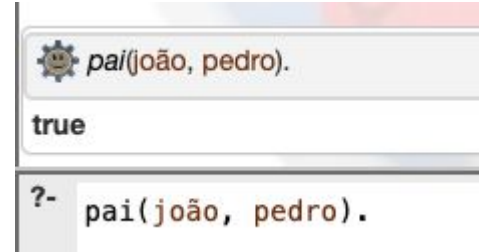
Exemplo

→ Possíveis perguntas:

?- pai(joão, pedro).
?- mãe(maria, clara).
?- filho(pedro, X).
?- filha(clara, X).
?- avô(joão, X).



Questões (queries)





Exemplo

```
1 pai(joão, pedro).
2 pai(joão, henrique).
3 pai(pedro, clara).
4 pai(pedro, clarissa).
5
6 mãe(maria, pedro).
7 mãe(maria, henrique).
8 mãe(luisa, clara).
9 mãe(luisa, clarissa).
10
11 homem(joão).
12 homem(pedro).
13 homem(henrique).
14
15 mulher(clara).
16 mulher(clarissa).
17 mulher(luisa).
```

```
19 parente(X, Y) :- pai(X, Y).
20 parente(X, Y) :- mãe(X, Y).
21
22 filho(X, Y) :- parente(Y, X), homem(X).
23 filha(X, Y) :- parente(Y, X), mulher(X).
24
25 avô(X, Y) :- pai(X, Z), parente(Z, Y).
26 avó(X, Y) :- mãe(X, Z), parente(Z, Y).
```



E se mudássemos as Regras?



Conceitos básicos

- **Fatos (Facts)** - tipo mais simples de statement. É uma forma de representar uma relação entre objetos.

```
pai(joão, pedro).
```

- **Questões (Queries)** - É uma forma de obter informação de um programa lógico. Pergunta se uma certa relação existe entre objetos. A resposta pode ser **true (yes)** ou **false (no)**

```
| ?- pai(joão, pedro).  
true
```




Conceitos básicos

- **Variável (Variable)** - É uma forma de resumir várias queries. Uma query contendo uma variável pergunta se existe um valor para essa variável que faz a query ser uma consequência lógica do programa

```
| ?- pai(X, pedro), X = gustavo.  
false
```

- **Substituição** - É um conjunto finito (possivelmente vazio) de pares da forma $X_i = t_i$, onde X_i é uma variável e $X_i \neq X_j$ para cada $i \neq j$, e X_i não ocorre em t_j , para qualquer i e j .



Conceitos básicos

→ **Instância (Instance)** - A é uma instância de B se existe uma substituição θ que $A = B\theta$

`pai(joão, pedro)` é instância de `pai(X, pedro)`

→ **Regras (rules)** - Regras são statements da forma:

$$A \leftarrow B_1, B_2, \dots, B_n.$$

onde $n \geq 0$

`filho(X,Y) :- pai(Y,X), homem(X).`



Cláusulas de Horn

- **Cláusulas de Horn** são fórmulas da forma

$$p \text{ :- } q_1, q_2, \dots, q_n$$

`<cabeça da cláusula> :- <corpo da cláusula>`

- **Fatos** são cláusulas de Horn com o corpo vazio;
- Variáveis na cabeça de cláusulas são **quantificadas universalmente** e seu escopo é toda a cláusula.
- Variáveis que ocorrem apenas no corpo da cláusula são **quantificadas existencialmente**



Cláusulas de Horn

- **Queries** são cláusulas de horn de cabeça vazia. São um meio de extrair informação de um programa. As variáveis que ocorrem nas questões são **quantificadas existencialmente**.

```
| :- filho(X, joão), X = pedro.  
true
```

- Responder uma questão é determinar se a questão é uma **consequência lógica** do programa.
- Responder uma questão com variáveis é dar uma instânciação da questão (representada por uma **substituição** para as variáveis) que é inferível do programa.



Termos

Termos são as entidades sintáticas que representam os objetos

→ Constantes

Inteiros (base 10: 5, outras bases: 2'101, código ASCII: 0'A), Reais (2.0, -5.71) ou Átomos.

Átomos são:

- qualquer sequência de caracteres alfanuméricos começada por **letra minúscula**
- Qualquer sequência de **+ - * / \ ^ > < = ~ : . ? @ # \$ &**
- Qualquer sequência de caracteres entre **' '**
- **! ; [] { }**



Termos

- **Variáveis:** Qualquer sequência de caracteres alfanuméricos iniciada com letra maiúscula ou _
X A _ _alfa _9 _val VAL Val
- **Termos compostos:** São objetos estruturados da forma $f(t_1, \dots, t_n)$ em que f é o functor do termo e $t_1, \dots, t_n$ são termos (subtermos). O functor f tem **aridade n** .
O nome do functor é um átomo.
data(6, 12, 23) suc(0) pred(5)

Funtores podem ser declarados como operadores: $+(X, Y) \Rightarrow X + Y$



Listas

Listas são termos gerados a partir do átomo `[]` (lista vazia) e do functor `. (H, T)`
`. (1, . (2, . (3, [])))` representa a lista `[1, 2, 3]`

O prolog tem uma notação especial para listas `[ head | tail ]`
`. (1, . (2, . (3, []))) = [1 | [2, 3]] = [1, 2 | [3]] = [1, 2, 3 | []]
= [1, 2, 3]`

→ **String:** é a lista de códigos ASCII dos seus caracteres.

`"PROLOG" = [80, 82, 79, 76, 79, 71]`

`"PROLOG"` é diferente de `'PROLOG'`



Listas

```
Sublist(X, Y) :- append(_, Z, Y), append(X, _, Z).
```

```
append([], L, L).
```

```
append([H|T], L, [H|T1]) :- append(T, L, T1).
```

% solução alternativa abaixo:

```
sublist(X, Y) :- append(_, X, _, Y).
```

```
append([], [], L, L).
```

```
append([], [H|T], L, [H|T1]) :- append([], T, L, T1).
```

```
append([H|T], X, L, [H|T1]) :- append(T, X, L, T1).
```




Programas

- Um programa é um conjunto de **cláusulas de Horn**
- Fatos (facts) e regras (rules) com o mesmo nome (cabeça da cláusula) definem um **predicado** (ou **procedimento**)

```
pai(joão, pedro).    pai(joão, henrique).  
homem(pedro).  mulher(clara).  
filho(X,Y) :- pai(Y,X), homem(X).  
filha(X,Y) :- pai(Y,X), mulher(X).
```

- Nesse exemplo definimos os predicados `pai/2`, `homem/1`, `mulher/1`, `filho/2` e `filha/2` (nome/aridade). É possível ter predicados distintos com mesmo nome e aridade diferente.



Operadores

- Prolog permite declarar funtores primários ou binários como operadores prefixos, posfixos ou infixos, associando-lhes ainda uma precedência.

`:= op(precedência, tipo, nome)`

precedência: 1 a 1200

nome: Functor ou lista de funtores

tipo:

associativo à esquerda

associativo à direita

não associativo

infixo

prefixo

posfixo

yfx

yf

xfy

fy

xfx

fx

xf

`:= op(500, yfx, [+,-])`

`:= op(300, fy, nao)`

`:= op(400, yfx, *)`



Operadores built-in

- Operador de unificação = .
- Teste e conversão de tipos: `atom`, `integer`, `real`, `var`, `name`, etc.
- Controle de busca: `!`, `fail`, `true`, `repeat`
- Negação por falha: `not`
- Operadores aritméticos: `is`, `+`, `-`, `*`, `/`, `rem`, `mod`, `^`, `**`, etc.
- Operadores aritméticos de comparação: `<`, `>`, `=<`, `>=`, `:=`, `=\=`.
- Entrada/Saída: `read`, `write`, `consult`, etc.
- Meta-programação: `assert`, `retract`, `call`
- Conjuntos `is_set(+Set)`, `is_list(+List)`, `list_to_set(+List, -Set)`, `subset(+Subset, +Set)`, `union(+Set1, +Set2, -Set3)`, etc



Operador de unificação = .

- O operador infixo = estabelece uma relação de unificação entre dois termos.
 $T1 = T2$ sucede se o termo $T1$ unifica com o termo $T2$
- Dois termos $T1$ e $T2$ unificam se existir uma substituição θ que aplicada aos termos $T1$ e $T2$ os torne literalmente iguais. Isto é, $T1\theta == T2\theta$.
- Dois átomos (ou números) só unificam se forem exatamente iguais.
- Dois termos compostos unificam se os funtores principais dos dois termos forem iguais e com a mesma aridade, e os argumentos respectivos unificam.
- Uma variável unifica com qualquer termo (gerando a substituição respectiva).



Unificação em listas

`[X|Y]` ou `[X|[Y|Z]]` unificam com `[a,b,c,d]`.

`[X,Y,Z]` não unifica com `[a,b,c,d]`

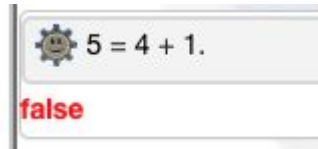
As consultas em Prolog:

<code>:- [X Y] = [a, b, c].</code>	<code>X=a Y=[b, c]</code>
<code>:- [X, Y, Z] = [a, b, c, d].</code>	<code>false</code>
<code>:- [X [Y Z]] = [a, b, c, d].</code>	<code>X=a Y=b Z=[c, d]</code>
<code>:- [X , Y Z] = [a, b, c, d].</code>	<code>X=a Y=b Z=[c, d]</code>

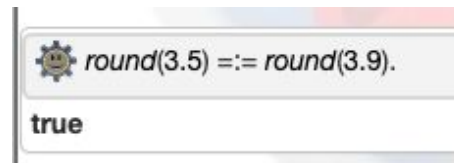


Operadores aritméticos

- Operadores aritméticos: $+$, $-$, $*$, $/$, `rem`, `mod`, $^$, $**$, etc.
- São **operadores simbólicos**, permitem construir expressões aritméticas, mas não efetuam qualquer cálculo



- O cálculo é efetuado utilizando os operadores aritméticos de comparação: $<$, $>$, $=$, $>=$, $=:=$, $=\backslash=$. Comparam os valores das expressões numéricas.
- **Z is** expressão: A expressão é calculada e seu resultado unifica com Z.





Operadores aritméticos

$X = Y$, $X \text{ is } Y$ e $X ::= Y$ são diferentes.

```
:- X = 1 + 2.
```

```
resp: X = 1 + 2
```

```
:- X is 3/2, Y is 3 mod 2.
```

```
resp: X=1.5 Y=1
```

```
:- 1+2 ::= 2+1.
```

```
resp: true
```

```
:- 1+A = B+2.
```

```
resp: A=2 B=1
```

```
:- Y ::= 1.
```

```
ERROR: ::= /2:
```

```
:- N=2, M=1, N ::= M+1.
```

```
resp: N = 2, M = 1.
```

```
:- X is 1 + 2.
```

```
resp: X = 3
```

```
:- 277 * 37 > 10000.
```

```
resp: true
```

```
:- 1+2 = 2+1.
```

```
resp: false
```

```
: - X=2, Y is X+2, X =\= Y , Z = 8
```

```
resp: X = 2, Y = 4, Z = 8.
```

```
:- X=2, Y is X+2, X ::= Y.
```

```
resp: false
```



Operadores aritméticos

```
1  fib(0,0).
2  fib(1,1).
3  fib(N,X) :- N > 1, N1 is N-1, fib(N1,A), N2 is N-2, fib(N2,B), X is A+B.
```

 `fib(5,X).`

X = 8

Next 10 100 1,000 Stop

 `time(fib(5, Fib)).`

26 inferences, 0.000 CPU in 0.000 seconds (88% CPU, 1469923 Lips)

Fib = 8

Next 10 100 1,000 Stop



Programação em lógica

- O formalismo lógico-computacional é fundamentado em três princípios básicos:
- ◆ Uso de **linguagem formal** para representação de conhecimento
 - ◆ Uso de **regras de inferência** para manipulação de conhecimento
 - ◆ Uso de uma **estratégia de busca** para controle de inferências



Programação em lógica

- O programador se preocupa somente em descrever o problema a ser solucionado: **especificação**



PROLOG

- Fragmento da lógica de 1ª ordens (**cláusulas de Horn**) para representar o conhecimento sobre um dado problema
- **Programa** = axiomas + regras de inferência (definindo relações entre objetos) que descrevem um dado problema.
- Chamamos esse conjunto de **base de conhecimento**.
- A **execução** de um programa consiste na dedução de consequências lógicas da base de conhecimento.
- O mecanismo de **inferência** é o algoritmo de **resolução de 1ª ordem**.



Estratégias de prova do motor de inferência do Prolog

- Regra de inferência é uma manipulação sintática que:
 - ◆ Permite criar novas fórmulas a partir de outras existentes
 - ◆ Em geral, simulam formas de raciocínio válidas
- Modus ponens:
 - ◆ Se vejo o sol, é dia. Vejo o sol. Logo, é dia

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$



Estratégias de prova do motor de inferência do Prolog

Dado: $?- G1, G2, \dots, Gn$

Busca a base de cima para baixo até unificar $G1$.

$\theta = \text{MGU tal que } C\theta = G1\theta$

- Fato F
- Regra $C :- P1 \dots Pm$

novo goal: $?- G2\theta, \dots, Gn\theta$

novo goal: $?- P1\theta, \dots, Pm\theta, G2\theta, \dots Gn\theta$



Estratégias de prova do motor de inferência do Prolog

→ Goal: ?- filha(clara, pedro)



Estratégias de prova do motor de inferência do Prolog

- Goal: `?- filha(clara, pedro)`
- `filha(clara, pedro)` **unifica com** `filha(X,Y) :- pai(Y,X), mulher(X),`
com θ : `{X= clara, Y= pedro}`



Estratégias de prova do motor de inferência do Prolog

→ Goal:

- ◆ `pai(pedro, clara)`
- ◆ `mulher(clara)`



Estratégias de prova do motor de inferência do Prolog

→ Goal:

◆ `pai(pedro, clara)`

◆ `mulher(clara)`

→ `pai(pedro, clara)` **unifica com o fato** `pai(pedro, clara)`



Estratégias de prova do motor de inferência do Prolog

→ Goal:

◆ `mulher(clara)`



Estratégias de prova do motor de inferência do Prolog

→ Goal:

◆ `mulher(clara)`

→ `mulher(clara)` **unifica com o fato** `mulher(clara)`



Estratégias de prova do motor de inferência do Prolog

→ Goal:
não há mais nada a provar

true



Estratégias de prova do motor de inferência do Prolog - Resumo

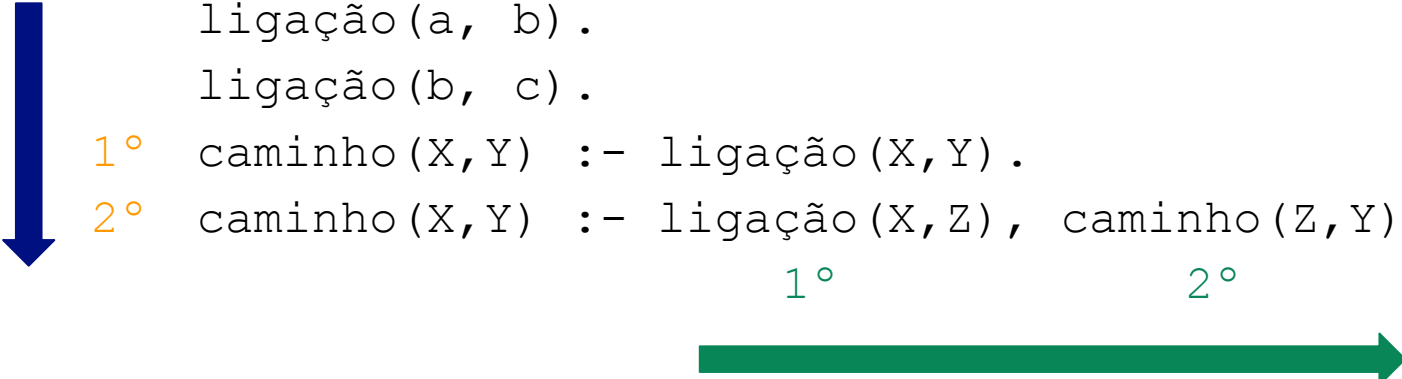
- Goal: `?- filha(clara, pedro)`
- `filha(clara, pedro)` **unifica com** `filha(X,Y) :- pai(Y,X), mulher(X),`
com θ : `{X= clara, Y= pedro}`
- Queremos provar agora os novos goals:
 - ◆ `pai(pedro, clara)`
 - ◆ `mulher(clara)`
- `pai(pedro, clara)` **unifica com o fato** `pai(pedro, clara)`
- `mulher(clara)` **unifica com o fato** `mulher(clara)`
- não há mais nada a provar



Estratégias de procura de soluções

- **Top Down**: de cima para baixo
- **Depth-first**: profundidade máxima antes de tentar encontrar um novo ramo
- **Backtracking**: volta a tentar encontrar uma prova alternativa

```
#1      ligação (a, b) .  
#2      ligação (b, c) .  
#3      1º caminho (X, Y) :- ligação (X, Y) .  
#4      2º caminho (X, Y) :- ligação (X, Z) , caminho (Z, Y) .  
      1º                               2º
```



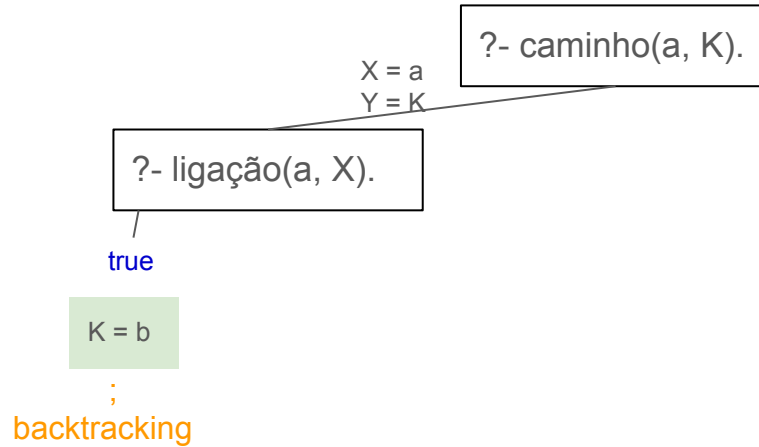


Estratégias de procura de soluções

?- caminho(a, K).

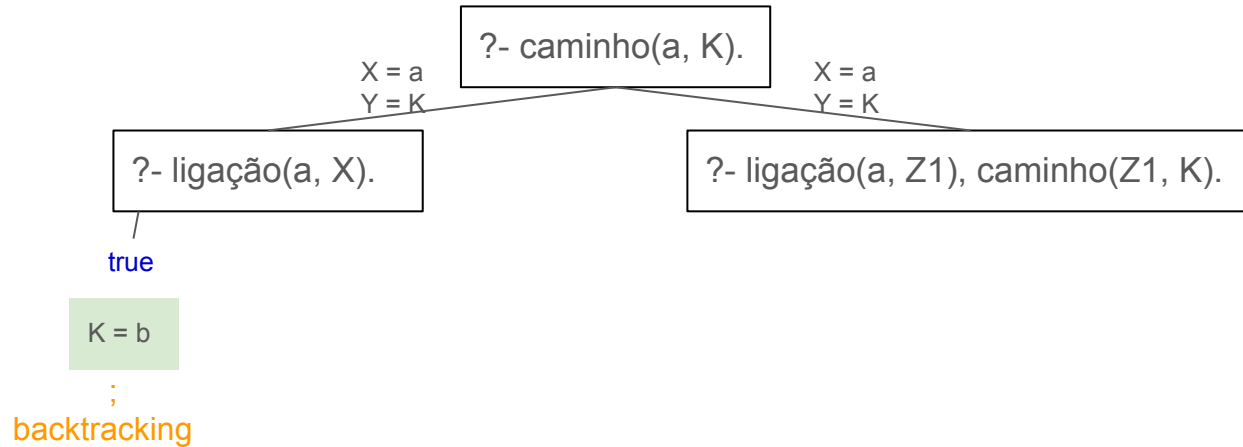


Estratégias de procura de soluções



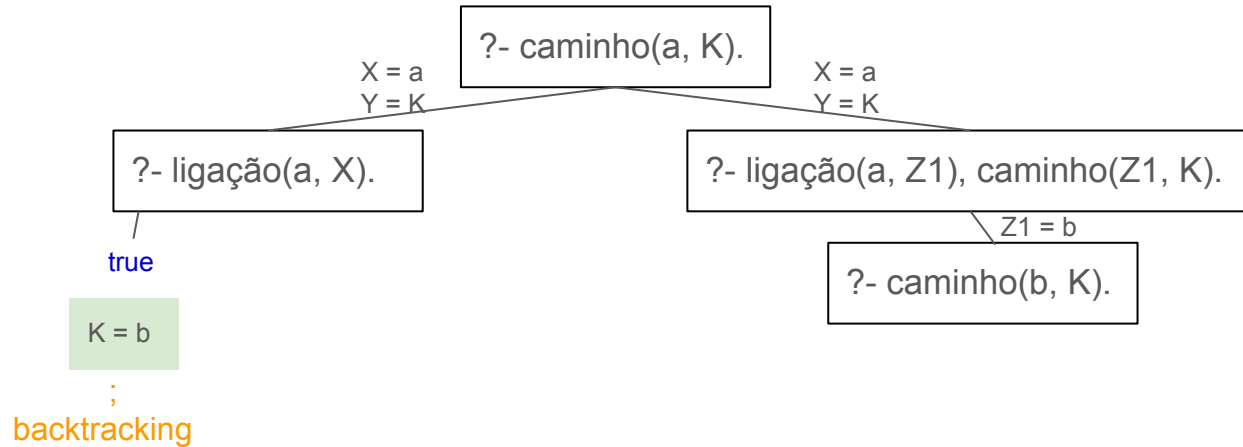


Estratégias de procura de soluções



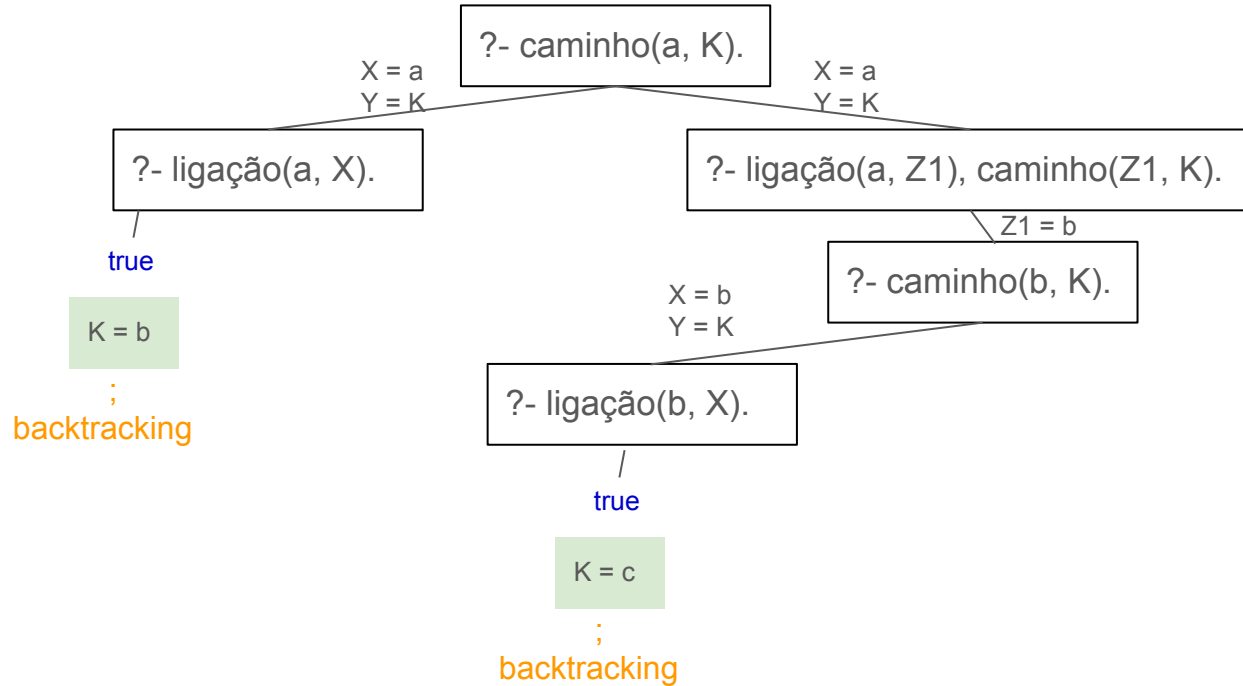


Estratégias de procura de soluções



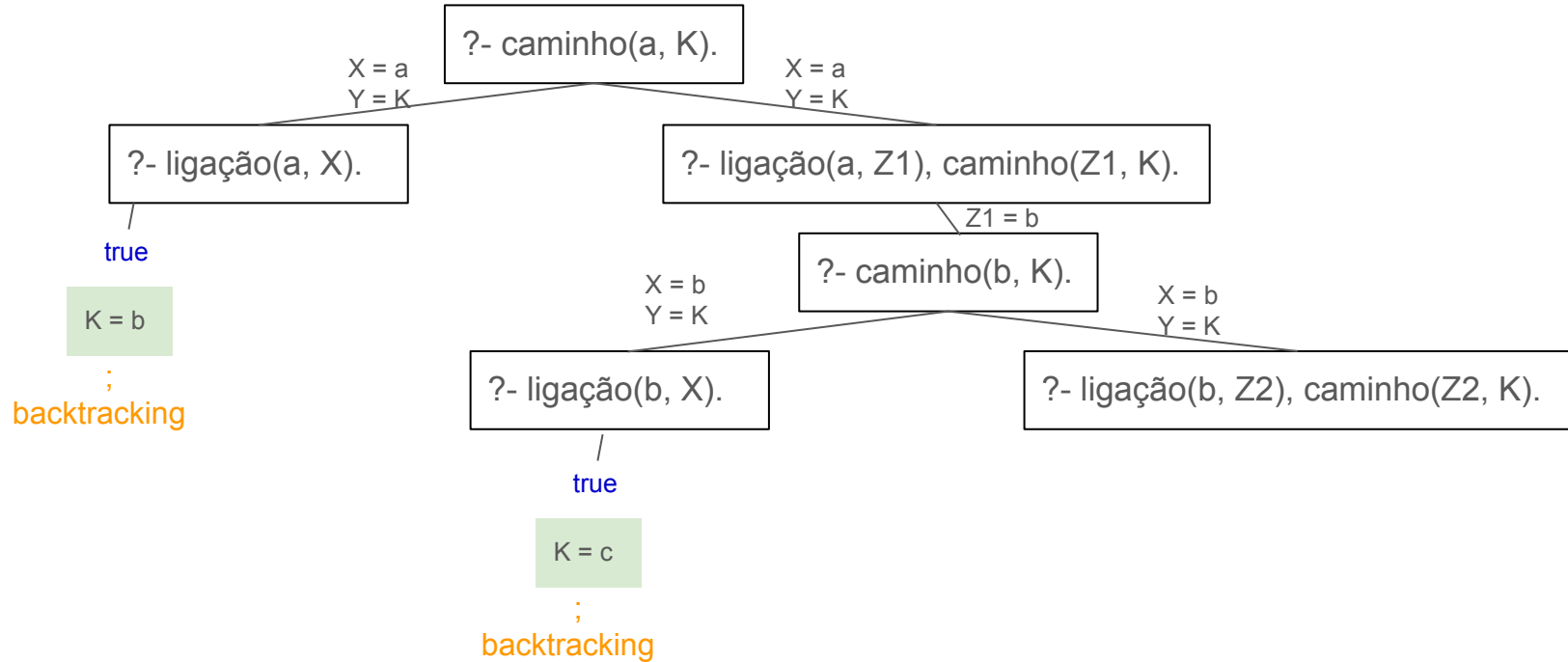


Estratégias de procura de soluções



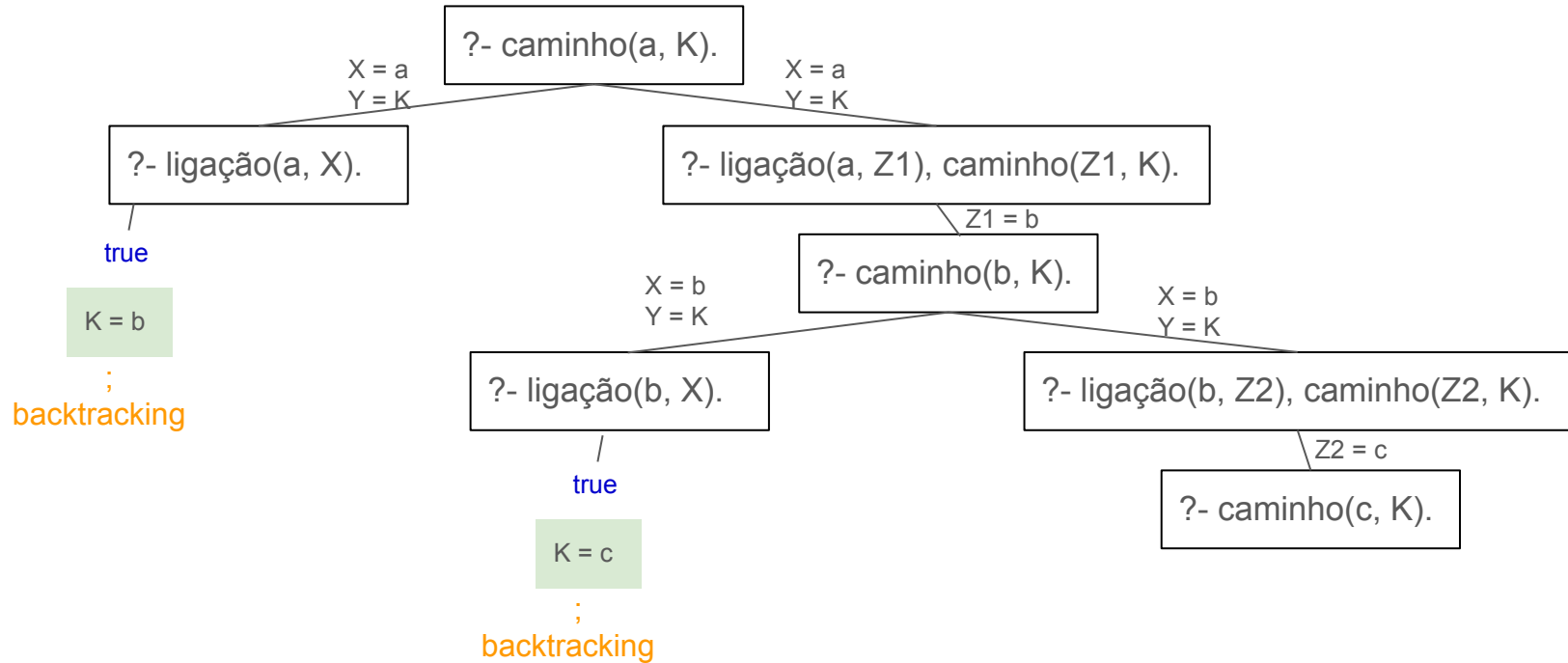


Estratégias de procura de soluções



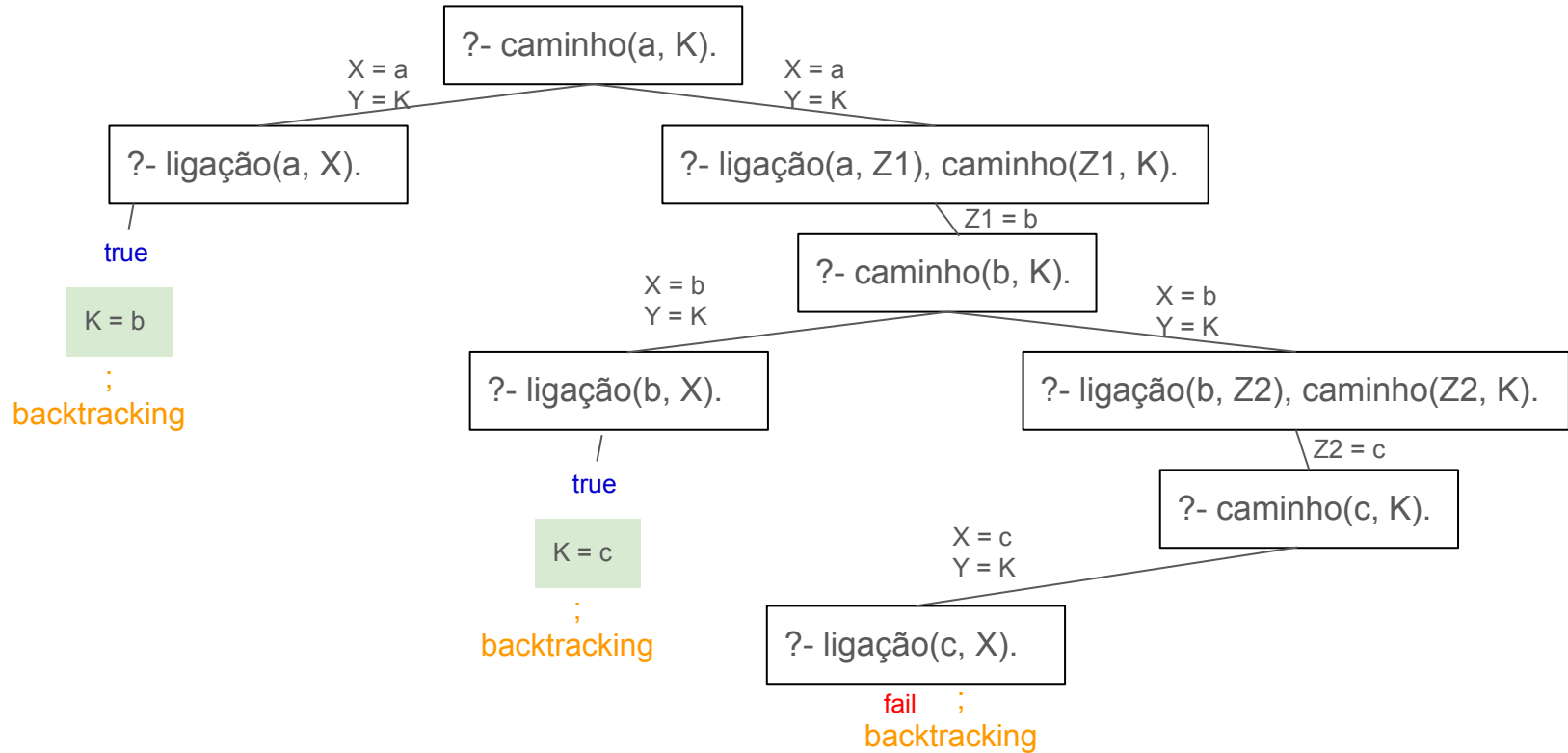


Estratégias de procura de soluções



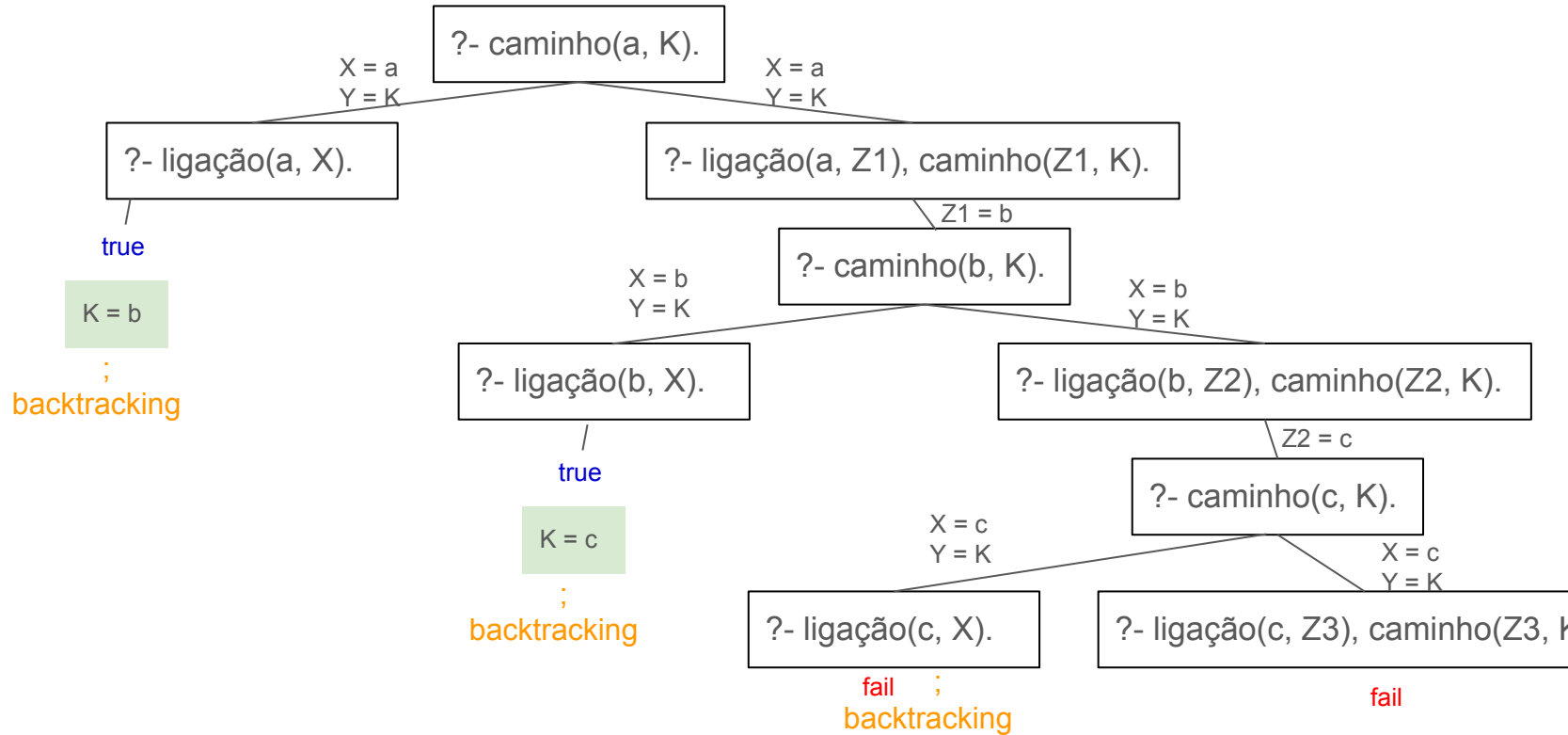


Estratégias de procura de soluções





Estratégias de procura de soluções





Exemplo: Banco de dados de filmes

<https://swish.swi-prolog.org/example/movies.pl>

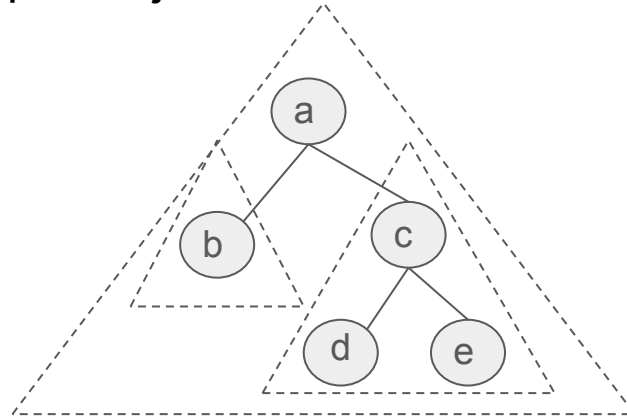
→ Que perguntas podemos fazer?

```
?- movie(american_beauty, Y).  
?- movie(M, 2000).  
?- movie(M, Y), Y < 2000.  
?- movie(M, Y), Y > 1999.  
?- actor(M1, A, _), actor(M2, A, _), M1 @> M2.  
?- actress(M, scarlett_johansson, _), director(M, D).  
?- actor(_, A, _), director(_, A).  
?- (actor(_, A, _) ; actress(_, A, _)), director(_, A).  
?- actor(M, john_goodman, _), actor(M, jeff_bridges, _).
```




Mais exemplos: Árvores binárias

- Uma árvore binária é uma estrutura que é ou vazia ou possui 3 componentes:
 - ◆ uma raiz (nó)
 - ◆ uma sub-árvore esquerda
 - ◆ uma sub-árvore direita
- A raiz pode ser qualquer objeto, mas as sub-árvores precisam ser árvores binárias





Mais exemplos: Árvores binárias

→ Podemos representar da seguinte forma:

- ◆ O átomo nil representa a árvore vazia
- ◆ Em prolog functor t representa uma árvore com raiz X, sub-árvore esquerda L e sub-árvore direita R da seguinte forma:
 $t(X, L, R)$.

```
:- t(a, t(b, nil, nil), t(c, t(d, nil, nil), t(e, nil, nil))).
```



Mais exemplos: Árvores binárias

- Podemos definir diversas funções.
- Ex: buscar se está na árvore

```
in(X,t(X,_,_)).
```

```
in(X,t(_,L,_)) :- in(X,L).
```

```
in(X,t(_,_,R)) :- in(X,R).
```



Mais exemplos: Árvores binárias

- Podemos definir diversas funções.
- Ex: Construir árvore binária de busca

```
add(X,nil,t(X,nil,nil)).
```

```
add(X,t(Root,L,R),t(Root,L1,R)) :- X @< Root, add(X,L,L1).
```

```
add(X,t(Root,L,R),t(Root,L,R1)) :- X @> Root, add(X,R,R1).
```

```
abb(L,T) :- abb(L,T,nil).
```

```
abb([],T,T).
```

```
abb([N|Ns],T,T0) :- add(N,T0,T1), abb(Ns,T,T1).
```



Operadores built-in

- Operador de unificação = .
- Testes e conversão de tipos: `atom`, `integer`, `real`, `var`, `name`, etc.
- Controle de busca: `!`, `fail`, `true`, `repeat`
- Negação por falha: `not`
- Operadores aritméticos: `is`, `+`, `-`, `*`, `/`, `rem`, `mod`, `^`, `**`, etc.
- Operadores aritméticos de comparação: `<`, `>`, `=<`, `>=`, `:=`, `=\=`.
- Entrada/Saída: `read`, `write`, `consult`, etc.
- Meta-programação: `assert`, `retract`, `call`
- Conjuntos `is_set(+Set)`, `is_list(+List)`, `list_to_set(+List, -Set)`, `subset(+Subset, +Set)`, `union(+Set1, +Set2, -Set3)`, etc



Testes e conversões de tipos (meta-lógica)

- Existem um conjuntos de predicados pré-definidos que nos permitem fazer uma análise dos termos: **atom**, **integer**, **real**, **var**, **name**, **etc**.
- **var(X)**
`?- var(X) .`
`true`
- **functor(+Term,?Name,?Arity)**
`?- functor(pai(X,pedro),N,A) .`
`N = pai, A = 2`
- **functor(?Term,+Name,+Arity)**
`?- functor(X,pai,2) .`
`X = pai(_A, _B)`



Testes e conversões de tipos (meta-lógica)

- ➔ `arg(+ArgNo,+Term,?Arg)`
`?- arg(2, pai(joão,pedro),A) .`
`A = pedro`
- ➔ `name(Átomo,Caracteres)`
`?- name(joão, X) .`
`X = [106, 111, 227, 111]`
- ➔ **Fato =.. Lista:**
`?- pai(joão, X) =.. P.`
`P = [pai, joão, X]`



Cut!

- O comando cut permite indicar ao Prolog quais sub-objetivos já satisfeitos não necessitam ser reconsiderados. Ele **aborta** o processo de **backtracking**.
- Roda **mais rápido**, sem perder tempo com sub-objetivos que não contribuem para determinar a resposta do objetivo principal.
- O programa ocupará **menos memória**, pois não será necessário armazenar todos os sub-objetivos (pontos do backtracking).
- O cut pode evitar que o programa entre em laço infinito.



Cut!

- A idéia é **podar** a árvore de prova para evitar ramos desnecessários e podendo tornar o programa **mais eficiente**.
- Altera o mecanismo de derivação de Prolog.
- Obriga o programador a saber como o programa será executado para poder entendê-lo.
- Comporta-se como uma fórmula atômica que é sempre satisfeita.
- É um comando extra lógico. Símbolos adicionais comuns: fail, repeat.



Cut!

- Durante o processo de prova, a 1ª passagem pelo cut é sempre verdadeira (com sucesso). Se por backtracking se voltar ao cut, então o cut faz falhar o predicado que está na cabeça da regra.
- Instrução imperativa (procedural) que vai contra a filosofia “declarativa” da programação em lógica.
(Não é uma boa técnica de programação)

ex:

```
x :- p, !, q.  
x :- r.
```

semelhante a: Se p então q else r



Negação por falha

- Não se pode afirmar que **not** é verdadeiro.
- Usa-se a hipótese do mundo fechado (close world assumption).
- **not** significa “Eu não posso provar que é verdadeiro”
- O prolog tem predefinido **true** e **fail**
- Utilizando **cut** e **fail** podemos implementar o **not**

```
:- op(700, fy, not)
not X :- X, !, fail
not X.
```



Negação por falha

→ No prolog o predicado de negação por falha é o `\+`

```
:- pai(luisa, clara)  
false
```

```
:- \+ pai(luisa, clara)  
true
```



Negação por falha

→ CUIDADO!

```
mulher(luisa) .  
casada(clara) .  
solteira(X) :- \+ casada(X), mulher(X) .
```





Negação por falha

→ CUIDADO!

```
mulher(luisa).  
casada(clara).  
solteira(X) :- mulher(X), \+ casada(X).
```



Meta predicados (programação de 2ª ordem)



- Existem meta-predicados que permitem colecionar todas as soluções para um dado:

`findall(?Template,:Goal,?Bag)`

- Bag é a lista de instâncias de Template encontradas nas provas de Goal. A ordem da lista corresponde à ordem em que são encontradas as respostas. Se não existirem instanciações para Template, Bag unifica com a lista vazia.

`bagof(?Template,:Goal,?Bag)`

- Semelhante a bagof, mas a lista é ordenada e sem repetições

`setof(?Template,:Goal,?Set)`



Dúvidas?

